

Práctica 1 — Plan de estudio

Duración: **1 hora**.

La interfaz ya está armada. Tu trabajo es darle comportamiento con `useReducer`, reunido en un custom hook. No agregues pantallas ni librerías.

Objetivo

Construir un plan de estudio pequeño en el que se puedan crear tareas, marcarlas, eliminarlas y filtrarlas. El formulario valida el título antes de agregar la tarea.

El estado compartido y todas sus actualizaciones viven en un solo custom hook. Los componentes solo reciben lo que ese hook expone.

Antes de empezar

```
npm install
npm run dev
```

Abre la dirección que muestra Vite. Vas a ver el formulario, el resumen, los filtros y dos tareas de ejemplo. Esas dos tareas son solo una maqueta visual: no viven en ningún estado.

Dónde va el estado

Crea un custom hook, por ejemplo `useStudyPlan`, en `src/hooks/useStudyPlan.ts`. Ese hook es el único lugar donde se usa `useReducer`.

El reducer guarda, en un solo estado:

- las tareas
- el filtro activo
- el mensaje de error del formulario, vacío cuando no hay error

El custom hook devuelve ese estado y las actualizaciones que los componentes van a compartir: agregar, completar, eliminar y cambiar el filtro. Los conteos se calculan a partir de la lista.

`App` llama el custom hook una sola vez y reparte esos valores. `TaskForm`, `TaskList`, `TaskItem`, `TaskFilters` y `TaskSummary` no llaman `useReducer` ni el custom hook. Siguen siendo componentes de presentación.

Firma, para consultar mientras trabajas:

```
const [state, dispatch] = useReducer(reducer, initialState)
```

- `useReducer`
- Reutilizar lógica con custom hooks

Comportamiento esperado

Lista

El estado inicial tiene la lista vacía, el filtro en todas y el mensaje de error vacío.

Cada tarea guarda:

- un `id` único
- el `title` que escribió la persona
- la `subject`, con uno de estos valores: `programacion`, `matematicas`, `historia`
- `done`, que empieza en `false`

Desde la interfaz se debe poder:

1. Agregar una tarea válida.
2. Marcar y desmarcar una tarea. El checkbox tiene que reflejar `done`.
3. Eliminar una tarea.
4. Cambiar el filtro entre todas, pendientes y completadas. El botón activo usa la clase `is-active`.
5. Ver el resumen: total, pendientes y completadas. Esos números se calculan a partir de la lista; no hace falta guardarlos como otro estado.

Quita las dos tareas de ejemplo de `TaskList` y pinta las tareas del estado. Si el filtro activo no tiene tareas, muestra el texto **No hay tareas para este filtro**. Puedes usar la clase `empty-state`, que ya está en el CSS.

Formulario

El formulario ya tiene los campos `title` y `subject`. Al enviarlo, la actualización sale del custom hook y llega al reducer.

Reglas:

- El título, sin espacios al inicio o al final, es obligatorio y debe tener al menos 3 caracteres.
- Si no cumple, no se agrega la tarea y el mensaje **Escribe un título de al menos 3 caracteres** aparece en la zona del formulario.
- Si la tarea es válida, el reducer la agrega a la lista, el mensaje de error desaparece y los campos quedan vacíos. La materia puede volver a Programación.

Dónde colocar la lógica

- El reducer y todas las actualizaciones están dentro del custom hook.
- `App` solo llama el hook y pasa a cada componente el estado y las actualizaciones que le tocan.
- Los componentes de la interfaz no declaran el estado de la lista, del filtro ni del error.

Los estilos están en `src/index.css` y se cargan desde `src/main.tsx`. No hace falta crear otra hoja de estilos.

Restricciones

- No uses `useState` para la lista, el filtro ni el mensaje de error.
- No llames `useReducer` fuera del custom hook.
- No guardes las tareas en `localStorage`.
- No instales dependencias nuevas.
- Para el id puedes usar `crypto.randomUUID()`.

Orden sugerido

1. Crea el custom hook y define el estado inicial y las acciones del reducer.
2. Haz que `App` use el hook y reemplace la maqueta de `TaskList` por la lista del estado.
3. Conecta completar y eliminar con las actualizaciones que devuelve el hook.
4. Conecta los filtros y el resumen.
5. Valida el formulario en el reducer y muestra el error.
6. Cuando el título sea válido, agrega la tarea desde el hook y limpia el formulario.

Listo cuando

- El estado compartido y sus actualizaciones están en un custom hook, y los componentes solo lo consumen.
- Al recargar, no aparece ninguna tarea.
- Un título vacío o de menos de 3 caracteres muestra el error y no agrega nada.
- Un título válido aparece en la lista y el formulario queda listo para otra tarea.
- Completar, eliminar y filtrar actualizan la lista y los tres conteos.
- El filtro activo se distingue con `is-active`.
- Una lista filtrada vacía muestra **No hay tareas para este filtro.**